JFlex = Crea analizador léxico para java

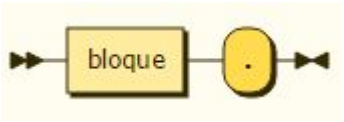
http://bottlecaps.de/rr/ui

()* 0 o más
()+ 1 o más veces
()? 0 o 1 vez

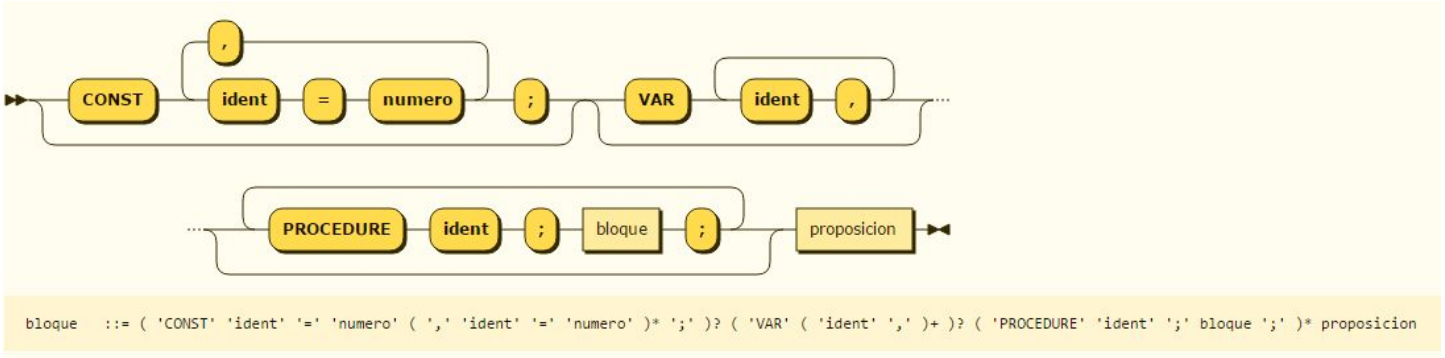
Kernighan Ritchie (Pagína 258 del libro, 135 del pdf): BNF que posee el lenguaje C

Si esta entre comillas es un símbolo terminal
Si además está en mayúscula es una palabra reservada

Diagrama de Sintáxis (DS):



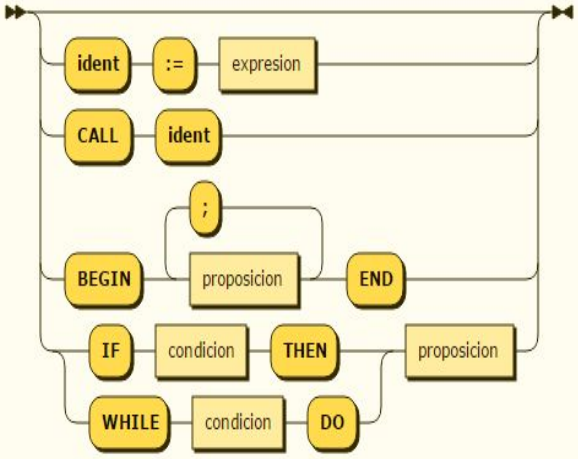
BNF: <programa>::=<bloque>
EBNF: programa ::= bloque '.'
bloque ::= ….




```
bloque   ::= ( 'CONST' 'ident' '=' 'numero' ( ',' 'ident' '=' 'numero' )* ';' )? ( 'VAR' ( 'ident' ',' )+ )? ( 'PROCEDURE' 'ident' ';' bloque ';' )* proposicion
```

bloque ::= ('CONST' 'ident' '=' 'numero' (',' 'ident' '=' 'numero')* ';')? ('VAR' ('ident' ',')+)? ('PROCEDURE' 'ident' ';' bloque ';')* proposicion

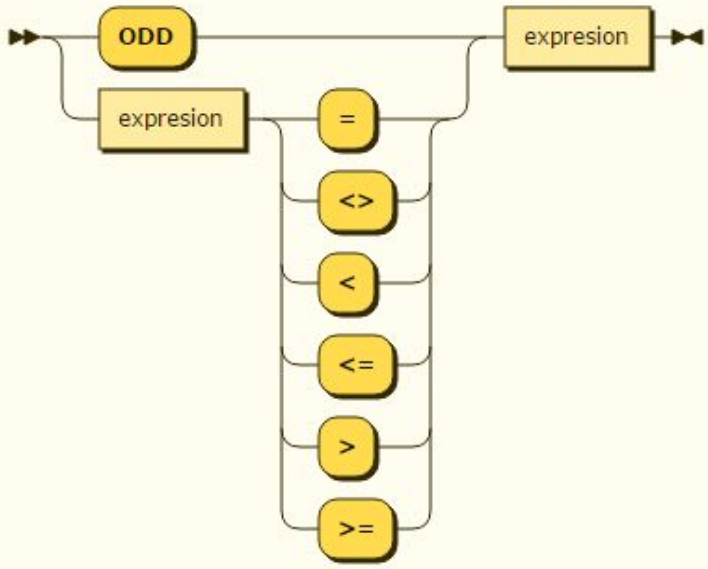
proposicion:




```
proposicion
        ::= ( 'ident' ':=' expresion | 'CALL' 'ident' | 'BEGIN' proposicion ( ';' proposicion )* 'END' | ( 'IF' condicion 'THEN' | 'WHILE' condicion 'DO' ) proposicion )?
```

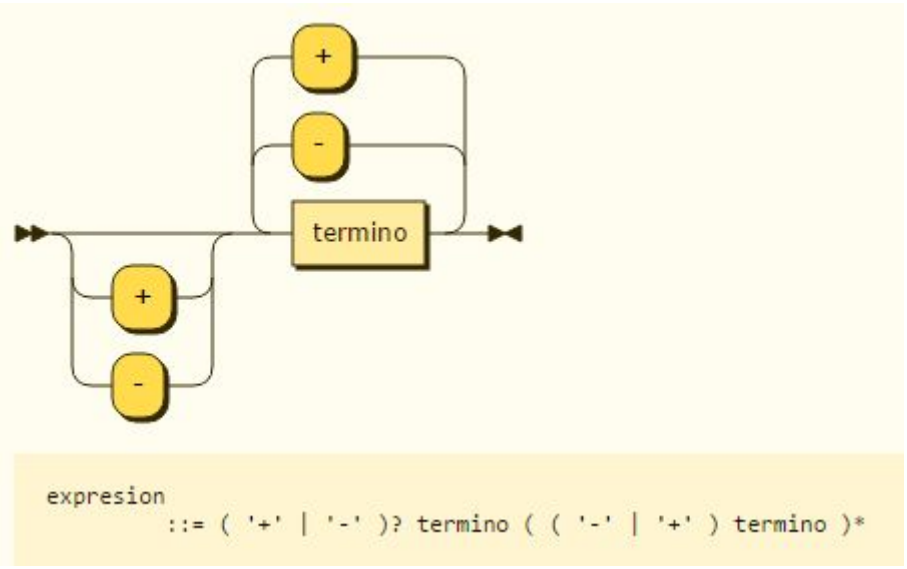
proposicion ::= ('ident' ':=' expresion | 'CALL' 'ident' | 'BEGIN' proposicion (';' proposicion)* 'END' | ('IF' condicion 'THEN' | 'WHILE' condicion 'DO') proposicion)?

condicion:



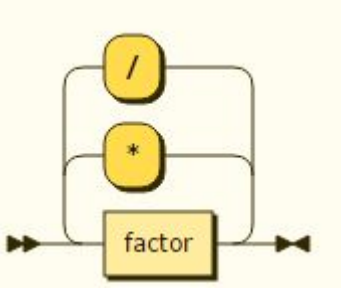

```
condicion
        ::= ( 'ODD' | expresion ( '=' | '<>' | '<' | '<=' | '>' | '>=' ) ) expresion
```

condicion ::= ('ODD' | expresion ('=' | '<>' | '<' | '<=' | '>' | '>=')) expresion

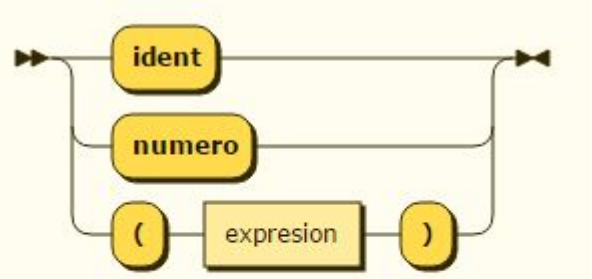



```
expresion
        ::= ( '+' | '-' )? termino ( ( '-' | '+' ) termino )*
```

expresion ::= ('+' | '-')? termino (('-' | '+') termino)*



termino ::= factor (('*' | '/') factor)*




```
factor   ::= 'ident'
           | 'numero'
           | '(' expresion ')'
```

factor ::= 'ident' | 'numero' | '(' expresion ')'

programa ::= bloque '.'
bloque::= ('CONST' (('ident' '=' 'numero') (',' 'ident' '=' 'numero')* ';'))?
('VAR' ('ident' ',')+)?
(('PROCEDURE' 'ident' ';' bloque ';')*)?
proposicion
proposicion ::= (('ident' ':=' expresion) | ('CALL' 'ident') | ('BEGIN' proposicion (';' proposicion)* 'END') | ('IF' condicion 'THEN' proposicion) | ('WHILE' condicion 'DO' proposicion))?
condicion ::= (('ODD' expresion) | (expresion ('=' | '<>' | '<' | '<=' | '>' | '>=') expresion))

expresion ::= ('+' | '-')? termino (('-' | '+') termino)*
termino ::= factor (((*') | ('/')) factor)*
factor ::= 'ident' | 'numero' | ('(' expresion ')')